

INFORME TÉCNICO: Arquitectura y Decisiones de Circle CI

Autor: Hernán Jurado Moran

Fecha: 2026-05-16

Proyecto: Escuela de Conducción - Pipeline CI/CD

Sistema: Circle CI 2.1 (Migración de Jenkins)

ÍNDICE

- Introducción
- Modularidad
- Reutilización de Plantillas
- Paralelismo en Tests
- Gestión de Ramas
- Decisiones de Arquitectura
- Conclusiones

1. INTRODUCCIÓN

Objetivo de la Migración

Migrar el pipeline de Jenkins (Groovy DSL) a Circle CI (YAML) para lograr:

- Simplicidad: YAML vs Groovy complejo
- Escalabilidad: Cloud-native vs servidor local
- Velocidad: Paralelismo nativo
- Mantenibilidad: Sintaxis estándar

Versión de Circle CI

version: 2.1 # Versión moderna con soporte para Orbs

2. MODULARIDAD

2.1 Estructura de Jobs

El pipeline se divide en 6 jobs independientes:

Job	Responsabilidad
checkout_and_build	Descargar código y compilar
unit_tests	Validar código individual
integration_tests	Validar estructura del proyecto
deploy_development	Deploy a ambiente DEV (si rama=develop)
deploy_production	Deploy a ambiente PROD (si rama=main)
report	Reporte final

2.2 Separación de Concerns

Cada job tiene UNA única responsabilidad clara y bien definida. Esto permite:

- Debugging independiente
- Ejecución paralela
- Mantenimiento simplificado

2.3 Comunicación entre Módulos

Problema: Los jobs corren en contenedores separados. ¿Cómo comparten archivos?

Solución: Workspace persistence con `persist_to_workspace` y `attach_workspace`

- En `checkout_and_build`: `persist_to_workspace` guarda artifacts
- En otros jobs: `attach_workspace` recupera artifacts

3. REUTILIZACIÓN DE PLANTILLAS

3.1 Patrón de Job Reutilizable

Problema: El mismo patrón se repite en múltiples jobs.

Solución: Crear estructura base reutilizable

3.2 Elementos Reutilizables (DRY Principle)

Docker Image

image: cimg/openjdk:21.0 # Reutilizado en 6 jobs

- Ventaja: Un cambio centralizado afecta a todos los jobs

Environment Variables

- APP_NAME = "jenkins-deber-demo"
- APP_VERSION = "1.0.\${CIRCLE_BUILD_NUM}"

Variables de Entorno Dinámicas

Variable	Valor Ejemplo	Uso
<code>\${CIRCLE_BUILD_NUM}</code>	42	Número del build
<code>\${CIRCLE_BRANCH}</code>	main	Rama actual
<code>\${CIRCLE_SHA1}</code>	abc123...	Commit SHA completo
<code>\${CIRCLE_SHA1:0:8}</code>	abc123ab	Primeros 8 caracteres
<code>\${CIRCLE_USERNAME}</code>	hernan	Usuario de Git

4. PARALELISMO EN TESTS

4.1 Problema Original (Jenkins)

En Jenkins, los tests corrían secuencialmente:

unit_tests (1s) → integration_tests (4s) → TOTAL: 5 segundos

4.2 Solución: Ejecución Paralela en Circle CI

En Circle CI, los tests corren SIMULTÁNEAMENTE:

checkout_and_build (2-3s)

- └─→ unit_tests (1s) EN PARALELO
- └─→ integration_tests (4s) EN PARALELO

→ report (1s)

TOTAL: 6-8 segundos (vs 5 secuencial)

4.3 Configuración del Paralelismo

La clave está en la sección workflows del config.yml:

- unit_tests requiere: checkout_and_build (pero NO integration_tests)
- integration_tests requiere: checkout_and_build (pero NO unit_tests)
- Esto permite ejecución simultánea

4.4 Ahorro de Tiempo

- Secuencial: Build 3s + Unit 1s + Integration 4s = 8s
- Paralelo: Build 3s + max(Unit 1s, Integration 4s) = 7s
- Ahorro: 1 segundo (12.5%)

4.5 Ventajas

- Velocidad: Menos tiempo total de ejecución
- Escalabilidad: Agregar más tests sin aumentar tiempo total
- Independencia: Si un test falla, el otro sigue ejecutando

5. GESTIÓN DE RAMAS

5.1 Estrategia de Ramas (GitFlow Simplificado)

- main → Rama de producción (protegida)
- develop → Rama de desarrollo
- feature/sprint-N-* → Ramas de características

5.2 Flujos Condicionales por Rama

- Deploy a PRODUCCIÓN: SOLO si rama = main
- Deploy a DESARROLLO: SOLO si rama = develop
- Tests: Se ejecutan en TODAS las ramas

5.3 Ejemplos de Ejecución por Rama

Rama: feature/sprint-5-auth

- Tests ejecutan
- NO hay deploy (rama != develop ni main)

Rama: develop

- Tests ejecutan
- Deploy a DEV ejecuta
- NO deploy a PROD

Rama: main

- Tests ejecutan
- Deploy a PROD ejecuta

6. DECISIONES DE ARQUITECTURA

6.1 ¿Por qué Circle CI y no Jenkins?

Aspecto	Jenkins	Circle CI
Sintaxis	Groovy (DSL complejo)	YAML (simple)
Servidor	Requiere servidor propio	Cloud-native
Setup	2-3 horas	5 minutos
Paralelismo	Manual y complejo	Nativo y simple
Costo	\$\$ (servidor + admin)	\$ (free tier)
Mantenimiento	Manual (actualizaciones)	Automático (SaaS)
Comunidad	Grande pero en decline	Moderna y activa

6.2 ¿Por qué 6 Jobs separados?

- Un solo job: Monolítico, sin paralelismo, difícil de debuggear
- 6 jobs: Modular, paralelo, mantenible, escalable

6.3 ¿Por qué persist_to_workspace?

- Permite compartir artifacts entre jobs sin clonar repo
- Más rápido y eficiente que clonar en cada job

7. CONCLUSIONES

7.1 Logros Alcanzados

- ✓ Modularidad: 6 jobs independientes
- ✓ Reutilización: Plantillas y variables compartidas
- ✓ Paralelismo: Tests simultáneos (ahorro de tiempo)
- ✓ Ramas: Deploys condicionales (develop → DEV, main → PROD)
- ✓ Simplicidad: YAML vs Groovy
- ✓ Cloud-native: Sin servidor local, 100% SaaS

7.2 Métricas del Pipeline

- checkout_and_build: 2-3 segundos
 - unit_tests: 1 segundo
 - integration_tests: 4 segundos (paralelo)
 - deploy: 3-5 segundos
 - report: 1 segundo
 - TOTAL: 8-10 segundos
-
- Tasa de éxito: 100%
 - Modularidad: 6 jobs independientes
 - Paralelismo: 2 jobs simultáneamente

- Documentación Relacionada
 - .circleci/config.yml - Configuración principal
 - .circleci/CONFIG_YML_ANOTADO.md - Anotaciones línea por línea
 - .circleci/GUIA_CONFIG_CIRCLECI.md - Guía técnica
 - jenkins-deber/tests/ - Scripts de testing

Documento preparado por: Hernán Jurado Moran

Para presentación en clase: Proyecto Titulación - UDLA

Fecha: 2026-05-16

Estado: ✓ COMPLETO